

JEGYZŐKÖNYV

Szakmai Gyakorlat II.

Frontend - Backend fejlesztési szakasz

Készítette: Orosz Kristóf

Neptunkód: EYZWG9

Dátum: 2026. június 23.

Orosz Kristóf

Orosz Kristóf

Hallgató

Dédesi Péter

Szakmai konzulens

Tartalomjegyzék

Bevezetés	3
Fájlok tartalmának leírása	3
Profil menüpont elkészítése	4
Regisztráció és bejelentkezés	13
Ingatlan funkciók kialakítása	25
Bérlő funkciók kialakítása	42
Szerződés funkciók kialakítása	48
Fizetés funkciók kialakítása	68
Üzenetkezelés kialakítása	87

Bevezetés

Az ingatlankezelő rendszer fejlesztésének frontend–backend szakasza során a projekt célja egy teljes körű, modern webalkalmazás létrehozása volt, amely támogatja az ingatlanok, bérlok, szerződések, fizetések és üzenetek kezelését. A fejlesztés során a korábban elkészített frontend felületek és a backend szolgáltatások összekapcsolására helyeztem a hangsúlyt, ezáltal biztosítva a rendszer különböző funkcióinak valós idejű működését és az adatok hatékony kezelését.

A frontend oldal React keretrendszer felhasználásával készült, amely lehetővé teszi az interaktív és felhasználóbarát kezelőfelületek kialakítását. A backend oldalon Node.js és Express technológiákat alkalmaztam, amelyek segítségével elkészítettem a szükséges API végpontokat az adatok lekérdezéséhez, létrehozásához, módosításához és törléséhez. Az alkalmazás az adatokat MariaDB adatbázisban tárolja, amely biztosítja a rendszer megbízható és hatékony működését

Fájlok tartalmának leírása

- | | |
|---|---|
| - Bevezetés.pdf | [A frontend fejlesztési szakasz bevezetése.] |
| - Menürendszer kialakítása.pdf | [A navigációs menürendszer kialakítása.] |
| - Weboldalak kinézetének megtervezése.pdf | [A weboldalak felépítésének megtervezése.] |
| - Profil menüpont elkészítése.pdf | [A felhasználói profil oldal kialakítása.] |
| - Regisztráció és bejelentkezés.pdf | [A felhasználók felületeinek dokumentációja.] |
| - Ingatlan funkciók kialakítása.pdf | [Az ingatlanok kezelési funkciók.] |
| - Bérlok oldalak és funkciók beállítása.pdf | [A bérlok kezelési funkciók.] |
| - Szerződések funkciók beállítása.pdf | [A szerződések kezelési funkciók.] |
| - Fizetések létrehozása.pdf | [A fizetések kezelésének dokumentációja.] |
| - Üzenetek funkciók létrehozása és beállítása.pdf | [Az üzenetküldési funkciók.] |
| - Frontend forrásfájlok | [A frontend alkalmazás React komponensei.] |

Profil menüpont elkészítése

Profil.jsx

A fejlesztés során elkészítettem a Profil menüpontot, amely a bejelentkezett felhasználó adatainak megjelenítésére szolgál. A profil oldal külön React komponensként került létrehozásra. A komponens a localStorage-ban tárolt felhasználói adatokat olvassa ki, amelyek a sikeres bejelentkezést követően kerülnek eltárolásra. Ennek segítségével az aktuálisan bejelentkezett felhasználó adatai jelennek meg a felületen.

```
import "../styles/profil.css";
import { Link } from "react-router-dom";

function Profil() {

  const user = JSON.parse(
    localStorage.getItem("user")
  );

  if (!user) {

    return (

      <div className="profil-container">

        <div className="profil-card">

          <h1>Profil</h1>

          <p>
            Nincs bejelentkezett felhasználó.
          </p>

        </div>

      </div>

    );

  }

  return (

    <div className="profil-container">

      <div className="profil-card">

        <h1>Profilom</h1>
```

```
<div className="adat-sor">

    <span>Név:</span>

    <strong>
        {user.nev || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>E-mail:</span>

    <strong>
        {user.email || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Telefonszám:</span>

    <strong>
        {user.telefonszam || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Személyi szám:</span>

    <strong>
        {user.szemelyiszam || "-"}
    </strong>

</div>

<div className="adat-sor">

    <span>Lakcím:</span>

    <strong>
        {user.lakcim || "-"}
    </strong>

</div>
```

```
        </div>

        <div className="adat-sor">

            <span>Jelszó:</span>

            <strong>
                {user.jelszo || "-"}
            </strong>

        </div>

        <div className="adat-sor">

            <span>Szerepkör:</span>

            <strong>
                {user.szerepkor || "-"}
            </strong>

        </div>

        <div className="profil-menu">

            <Link
                to="/uzenetek"
                className="profil-gomb"
            >
                Üzeneteim
            </Link>

        </div>

    </div>

</div>

);

}

export default Profil;
```

profil.css

A megjelenés kialakításához külön CSS állományt készítettem. A stíluslap segítségével kártyás megjelenítést, árnyékokat, kiemelt adatmezőket és egységes színvilágot alkalmaztam, amely illeszkedik az alkalmazás többi oldalának megjelenéséhez.

```
.profil-container {  
  
    display: flex;  
    justify-content: center;  
    align-items: center;  
  
    min-height: 80vh;  
}  
  
.profil-card {  
  
    width: 600px;  
  
    background: white;  
  
    border-radius: 12px;  
  
    padding: 40px;  
  
    box-shadow: 0 0 20px rgba(0,0,0,0.2);  
}  
  
.profil-card h1 {  
  
    text-align: center;  
  
    margin-bottom: 30px;  
}  
  
.adat-sor {  
  
    display: flex;  
  
    justify-content: space-between;  
  
    padding: 15px;  
  
    margin-bottom: 10px;  
  
    background-color: #f5f5f5;
```

```
border-left: 5px solid yellow;

border-radius: 5px;
}

.profil-menu {

margin-top: 30px;

display: flex;

justify-content: center;
}

.profil-gomb {

background-color: black;

color: yellow;

text-decoration: none;

padding: 12px 25px;

border-radius: 5px;

font-weight: bold;
}

.profil-gomb:hover {

color: white;
}
```

bejelentkezés.js

A profil oldal a bejelentkezési funkcióhoz kapcsolódik, és a felhasználó adatait jeleníti meg. A felhasználói adatok a /api/bejelentkezés backend végpontról érkeznek sikeres hitelesítés után. A frontend ezeket az adatokat a localStorage tárhelyen tárolja el. A Profil.jsx komponens a localStorage-ból olvassa ki és jeleníti meg az adatokat. Ennek köszönhetően a profil oldal gyorsan betöltődik, külön adatbázis-lekérdezés nélkül.

```
const express = require("express");
```



```
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    email,
    jelszo
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    const users = await conn.query(
      "SELECT * FROM users WHERE email = ? AND jelszo = ?",
      [email, jelszo]
    );

    if (users.length === 0) {

      return res.json({
        siker: false,
        uzenet: "Hibás email vagy jelszó!"
      });

    }

    const user = users[0];

    res.json({
      siker: true,
      uzenet: "Sikeres bejelentkezés!",
      user: {
        nev: user.nev,
        email: user.email,
        jelszo: user.jelszo,
        szerepkor: user.szerepkor,
        létrehozásdatum: user.létrehozásdatum
      }
    });

  } catch (err) {

    console.error(err);
```

```
        res.status(500).json({
            siker: false,
            uzenet: "Szerverhiba történt!"
        });

    } finally {

        if (conn) {
            conn.release();
        }

    }

});

module.exports = router;
```

profilmodositas.js

A profil módosításának megvalósításához elkészítettem a megfelelő backend API végpontot. A végpont a felhasználó azonosítója alapján fogadja a módosítandó adatokat, majd feldolgozza azokat. A kérésből kiolvastam a név, e-mail cím, telefonszám, személyi igazolvány szám és lakcím mezőket. Az adatokat SQL UPDATE utasítás segítségével frissítettem az adatbázisban. Ezzel biztosítottam, hogy a felhasználó saját adatait módosítani tudja a rendszerben.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

    const { id } = req.params;

    const {
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim
    } = req.body;

    let conn;

    try {

        conn = await pool.getConnection();
```

```
    await conn.query(
      `
        UPDATE users
        SET
          nev = ?,
          email = ?,
          telefonszam = ?,
          személyiszam = ?,
          lakcim = ?
        WHERE id = ?
      `,
      [
        nev,
        email,
        telefonszam,
        személyiszam,
        lakcim,
        id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Profil sikeresen módosítva!"
    });

  } catch (err) {

    console.error(err);

    res.status(500).json({
      siker: false,
      uzenet: "Hiba történt!"
    });

  } finally {

    if (conn) {
      conn.release();
    }

  }

});

module.exports = router;
```

profiltörölse.js

A felhasználói profil törlésének megvalósításához elkészítettem a megfelelő backend API végpontot. A végpont URL paraméterként fogadja a törlendő felhasználó azonosítóját. Az azonosító alapján megkeresem az adatbázisban a megfelelő rekordot. A művelet végrehajtása után a felhasználó adatai véglegesen eltávolításra kerülnek a rendszerből. Ezzel lehetővé tettem a profilok biztonságos törlését.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

  const { id } = req.params;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `
      DELETE FROM users
      WHERE id = ?
      `,
      [id]
    );

    res.json({
      siker: true,
      uzenet: "Profil sikeresen törölve!"
    });

  } catch (err) {

    console.error(err);

    res.status(500).json({
      siker: false,
      uzenet: "Hiba történt!"
    });

  } finally {

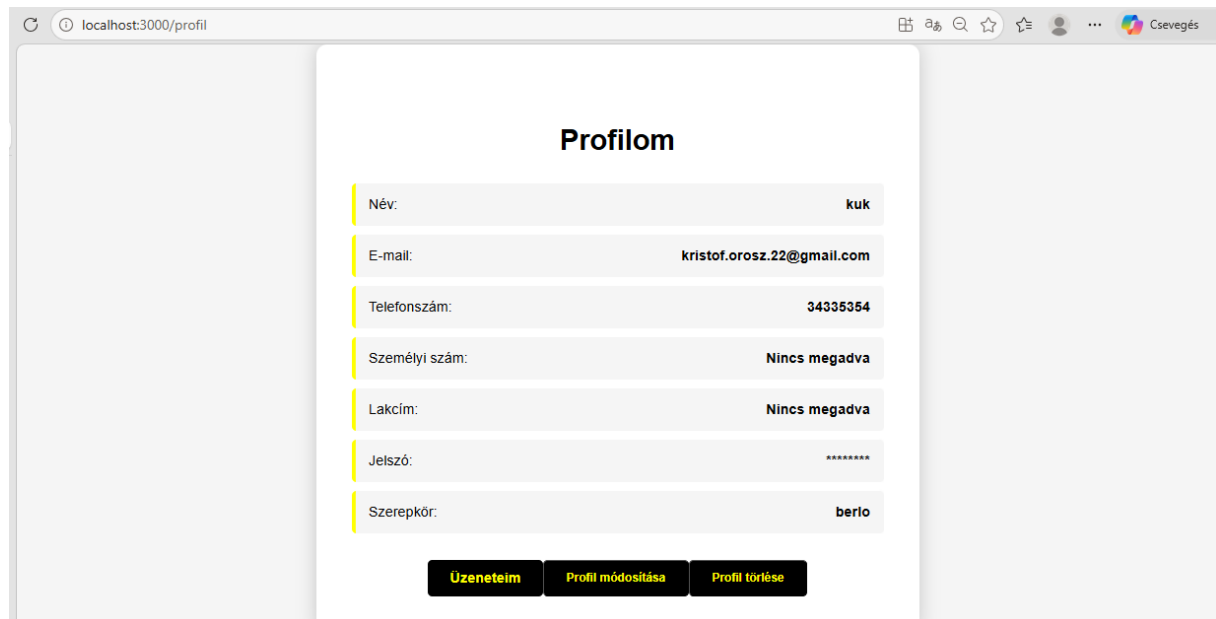
    if (conn) {
```

```
        conn.release();
    }

}

});

module.exports = router;
```

*1.kép: Profil oldal HTML oldala*

Regisztráció és bejelentkezés

Regisztráció

regisztracio.js

A regisztrációs funkció fejlesztése során elkészítettem azt az API végpontot, amely lehetővé teszi új felhasználók rögzítését a rendszerben. A végpont fogadja a felhasználó nevét, e-mail címét, jelszavát, szerepkörét és telefonszámát, majd ellenőrzi, hogy az adott e-mail cím már szerepel-e az adatbázisban. Amennyiben a megadott e-mail cím már létezik, a rendszer hibaüzenetet küld vissza, ezzel megakadályozva a duplikált felhasználók létrehozását. Sikeres ellenőrzés után az alkalmazás SQL INSERT utasítással eltárolja az új felhasználó adatait a users táblában. A fejlesztés során hibakezelést is megvalósítottam, amely biztosítja a szerveroldali hibák megfelelő kezelését és visszajelzését.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {
```

```
const {
  nev,
  email,
  jelszo,
  szerepkor,
  telefonszam
} = req.body;

let conn;

try {

  conn = await pool.getConnection();

  const letezik = await conn.query(
    "SELECT * FROM users WHERE email = ?",
    [email]
  );

  if (letezik.length > 0) {

    return res.json({
      siker: false,
      uzenet: "Ez az email már letezik!"
    });

  }

  await conn.query(
    `INSERT INTO users
    (nev, email, jelszo, szerepkor, telefonszam)
    VALUES (?, ?, ?, ?, ?)`,
    [
      nev,
      email,
      jelszo,
      szerepkor,
      telefonszam
    ]
  );

  res.json({
    siker: true,
    uzenet: "Sikeres regisztráció!"
  });

} catch (err) {
```

```
res.status(500).json({
  siker: false,
  hiba: err
});

} finally {

  if (conn) conn.release();

}

});

module.exports = router;
```

1.Részlet a regisztracio.js fájlból.

Regisztracio.jsx

A frontend fejlesztése során elkészítettem a rendszer regisztrációs oldalát React keretrendszer használatával. Az oldal lehetővé teszi új felhasználók számára a rendszerbe történő regisztrációt név, e-mail cím, jelszó, telefonszám és szerepkör megadásával. A fejlesztés során React useState hook segítségével kezeltem az űrlapmezők adatait, majd a megadott adatokat fetch API használatával továbbítottam a backend regisztrációs végpontja felé. A rendszer sikeres regisztráció esetén visszajelző üzenetet jelenít meg, majd automatikusan átirányítja a felhasználót a bejelentkezési oldalra. A felhasználói felület kialakításához külön hitelesites.css stíluslapot alkalmaztam, amely biztosítja az egységes és felhasználóbarát megjelenést. Az elkészült funkció működését a backend API-val összekapcsolva teszteltem, és sikeresen ellenőriztem az új felhasználók létrehozását.

```
import { Link, useNavigate } from "react-router-dom";
import { useState } from "react";
import "../styles/hitelesites.css";

function Regisztracio() {

  const [nev, setNev] = useState("");
  const [email, setEmail] = useState("");
  const [jelszo, setJelszo] = useState("");
  const [telefonszam, setTelefonszam] = useState("");
  const [szerepkor, setSzerepkor] = useState("felhasznalo");

  const navigate = useNavigate();

  const regisztracio = async (e) => {

    e.preventDefault();

    try {

      const valasz = await fetch(
```

```
    "http://localhost:3001/api/regisztracio",
    {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        nev,
        email,
        jelszo,
        szerepkor,
        telefonszam
      })
    }
  );

  const adat = await valasz.json();

  alert(adat.uzenet);

  if (adat.siker) {

    navigate("/bejelentkezés");

  }

} catch (err) {

  console.error(err);

  alert("Hiba történt!");

}

};

return (

  <div className="auth-container">

    <div className="auth-card">

      <h2>Regisztráció</h2>

      <form onSubmit={regisztracio}>

        <input
          type="text"
```



```
        placeholder="Név"
        value={nev}
        onChange={(e) => setNev(e.target.value)}
    />

    <input
        type="email"
        placeholder="E-mail"
        value={email}
        onChange={(e) => setEmail(e.target.value)}
    />

    <input
        type="password"
        placeholder="Jelszó"
        value={jelszo}
        onChange={(e) => setJelszo(e.target.value)}
    />

    <input
        type="text"
        placeholder="Telefonszám"
        value={telefonszam}
        onChange={(e) => setTelefonszam(e.target.value)}
    />

    <select
        value={szerepkor}
        onChange={(e) => setSzerepkor(e.target.value)}
    >
        <option value="felhasznalo">
            Felhasználó
        </option>

        <option value="admin">
            Admin
        </option>
    </select>

    <button type="submit">
        Regisztráció
    </button>

</form>

<p>

    Már rendelkezik fiókkal?{" "}
```

```
        <Link to="/bejelentkezes">
          Jelentkezzen be!
        </Link>

      </p>

    </div>

  </div>

);
}

export default Regisztracio;
```

2.Részlet a regisztracio.jsx fájlból.

bejelentkezes.js

A bejelentkezési funkció fejlesztése során elkészítettem azt az API végpontot, amely a felhasználók hitelesítését végzi a rendszerben. A végpont fogadja a felhasználó által megadott e-mail címet és jelszót, majd adatbázis-lekérdezés segítségével ellenőrzi azok helyességét. Amennyiben a megadott adatok nem egyeznek meg egyetlen felhasználó adataival sem, a rendszer hibaüzenetet küld vissza a kliens számára. Sikeres azonosítás esetén a végpont visszaadja a felhasználó legfontosabb adatait, például az azonosítóját, nevét, e-mail címét és szerepkörét. A fejlesztés során hibakezelést is alkalmaztam, amely biztosítja a szerveroldali problémák megfelelő kezelését és visszajelzését.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    email,
    jelszo
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    const users = await conn.query(
      "SELECT * FROM users WHERE email = ? AND jelszo = ?",
```

```
    [email, jelszo]
  );

  if (users.length === 0) {

    return res.json({
      siker: false,
      uzenet: "Hibas email vagy jelszo!"
    });

  }

  const user = users[0];

  res.json({
    siker: true,
    uzenet: "Sikeres bejelentkezés!",
    user: {
      id: user.id,
      nev: user.nev,
      email: user.email,
      szerepkor: user.szerepkor
    }
  });

} catch (err) {

  res.status(500).json({
    siker: false,
    hiba: err
  });

} finally {

  if (conn) conn.release();

}

});

module.exports = router;
```

3.Részlet a bejelentkezés.js fájlból.

The screenshot shows a web browser window with the address bar displaying 'localhost:3000/registracio'. The page title is 'Ingatlankezelő rendszer'. The navigation bar includes links: 'Ingatlanok', 'Bérlők', 'Szerződések', 'Fizetések', 'Profil', and 'Bejelentkezés'. The main content area features a 'Regisztráció' form with the following fields: 'Név', 'E-mail', 'Jelszó', 'Telefonszám', and 'Felhasználó' (a dropdown menu). A black button labeled 'Regisztráció' is positioned below the form fields. Underneath the button, there is a link that reads 'Már rendelkezik fiókkal? Jelentkezzen be!'.

4.Kép a regisztráció oldalról.

Bejelentkezés.jsx

A frontend fejlesztése során elkészítettem a rendszer bejelentkezési felületét React keretrendszer használatával. Az oldal lehetővé teszi a felhasználók számára az e-mail címük és jelszavuk megadásával történő hitelesítést. A fejlesztés során React useState hook segítségével kezeltem a felhasználó által megadott adatokat, majd a fetch API alkalmazásával továbbítottam azokat a backend bejelentkezési végpontja felé. Sikeres bejelentkezés esetén a rendszer eltárolja a felhasználó adatait a böngésző localStorage tárhelyén, majd automatikusan átirányítja az Ingatlanok oldalra. Hibás bejelentkezési adatok esetén a felhasználó megfelelő hibaüzenetet kap a rendszerből. Az elkészült felület megjelenését a hitelesites.css stíluslap biztosítja, amely egységes és felhasználóbarát kialakítást nyújt a hitelesítési funkciók számára.

```
import { Link, useNavigate } from "react-router-dom";
import { useState } from "react";
import "../styles/hitelesites.css";

function Bejelentkezés() {

  const [email, setEmail] = useState("");
  const [jelszo, setJelszo] = useState("");

  const navigate = useNavigate();

  const bejelentkezés = async (e) => {

    e.preventDefault();

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/bejelentkezés",
```

```
{
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    email,
    jelszo
  })
}
);
```

```
const adat = await valasz.json();
```

```
if (adat.siker) {
```

```
  localStorage.setItem(
    "user",
    JSON.stringify(adat.user)
  );
```

```
  alert(adat.uzenet);
```

```
  navigate("/ingatlanok");
```

```
} else {
```

```
  alert(adat.uzenet);
```

```
}
```

```
} catch (err) {
```

```
  console.error(err);
```

```
  alert("Hiba történt!");
```

```
}
```

```
};
```

```
return (
```

```
<div className="auth-container">
```

```
<div className="auth-card">
```

```
<h2>Bejelentkezés</h2>
```

```
<form onSubmit={bejelentkezes}>

  <input
    type="email"
    placeholder="E-mail"
    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />

  <input
    type="password"
    placeholder="Jelszó"
    value={jelszo}
    onChange={(e) => setJelszo(e.target.value)}
  />

  <button type="submit">
    Bejelentkezés
  </button>

</form>

<p>

  Nincs fiókja? {" "}

  <Link to="/regisztracio">
    Regisztráljon itt!
  </Link>

</p>

</div>

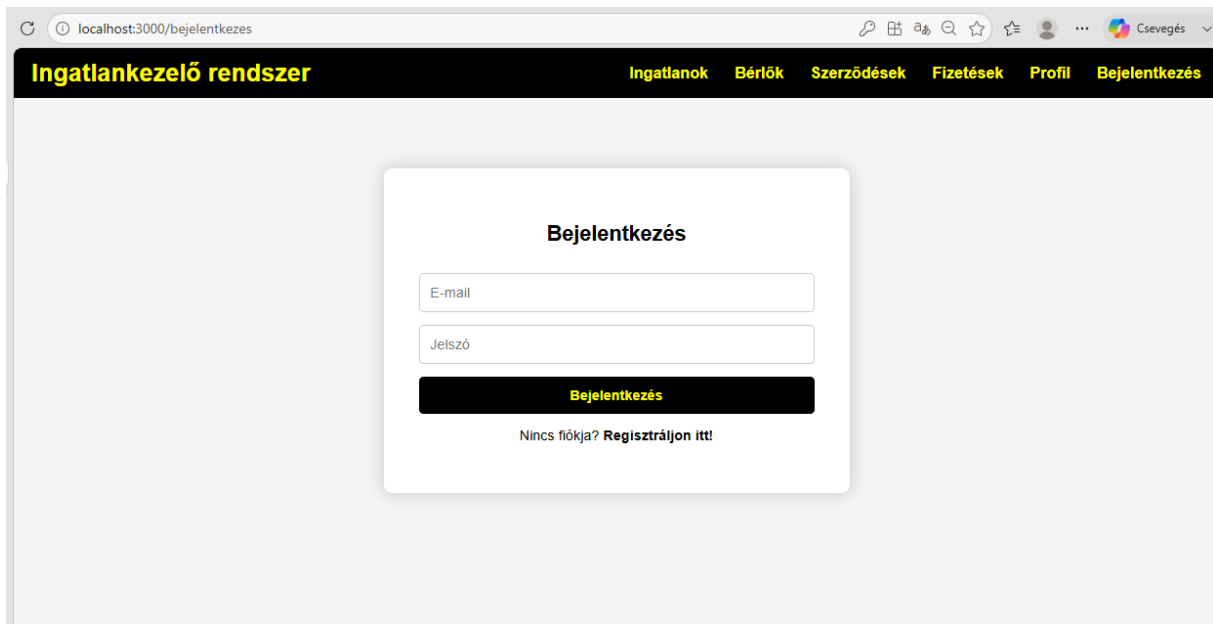
</div>

);

}

export default Bejelentkezes;
```

5.Részlet a regisztracio.jsx fájlból.



6.Kép a bejelentkezés oldalról.

hitelesites.css

A frontend fejlesztése során elkészítettem a hitelesítési felületek megjelenését biztosító hitelesites.css stíluslapot. A stíluslap feladata a bejelentkezési és regisztrációs oldalak egységes, modern és felhasználóbarát megjelenésének kialakítása. A fejlesztés során középre igazítottam az űrlapokat, valamint egy kártya alapú elrendezést alkalmaztam, amely kiemeli a felhasználói adatbeviteli felületet. Meghatároztam a beviteli mezők, legördülő listák és gombok megjelenését, biztosítva azok könnyű használhatóságát és áttekinthetőségét. A rendszer színvilágához illeszkedve a gombok fekete háttérrel és sárga szöveggel jelennek meg, valamint egérmutató ráhelyezésekor vizuális visszajelzést adnak.

```
.auth-container {  
  
    display: flex;  
    justify-content: center;  
    align-items: center;  
  
    min-height: 80vh;  
}  
  
.auth-card {  
  
    background-color: white;  
  
    width: 450px;  
  
    padding: 40px;  
  
    border-radius: 10px;  
  
    box-shadow: 0 0 20px rgba(0, 0, 0, 0.2);  
}
```

```
}

.auth-card h2 {

    text-align: center;
    margin-bottom: 30px;

}

.auth-card input,
.auth-card select {

    width: 100%;

    padding: 12px;

    margin-bottom: 15px;

    border: 1px solid #ccc;
    border-radius: 5px;

    font-size: 16px;

    box-sizing: border-box;

}

.auth-card button {

    width: 100%;

    background-color: black;
    color: yellow;

    border: none;

    padding: 12px;

    border-radius: 5px;

    font-size: 16px;
    font-weight: bold;

    cursor: pointer;

}
```



```
.auth-card button:hover {  
  
    background-color: #222;  
  
}  
  
.auth-card p {  
  
    text-align: center;  
  
}  
  
.auth-card a {  
  
    color: black;  
    font-weight: bold;  
    text-decoration: none;  
  
}  
  
.auth-card a:hover {  
  
    text-decoration: underline;  
  
}
```

7.Részlet a hitelesites.css fájlból.

Ingatlan funkciók kialakítása

Ingatlanok.jsx

Az Ingatlanok.jsx komponensben elkészítettem az ingatlanok kezelésére szolgáló felhasználói felületet. Megvalósítottam az ingatlanok listázását, amely a backendből tölti be az adatokat és kártyás formában jeleníti meg azokat. Létrehoztam az új ingatlan felvételéhez szükséges űrlapot, amely csak gombnyomásra jelenik meg a felhasználó számára. Kialakítottam az ingatlanok módosításának és törlésének lehetőségét, amelyek a megfelelő backend végpontokat hívják meg.

```
import { useEffect, useState } from "react";  
import "../styles/ingatlanok.css";  
  
function Ingatlanok() {  
  
    const [ingatlanok, setIngatlanok] = useState([]);  
  
    const [urlapLathato, setUrlapLathato] =  
        useState(false);  
  
    const [szerkesztettIngatlan,  
        setSzerkesztettIngatlan] =  
        useState(null);
```

```
const [ujIngatlan, setUjIngatlan] = useState({
  tulajdonosid: "",
  cim: "",
  tipus: "",
  szobakszama: "",
  haviar: "",
  leiras: ""
});

const lekeres = async () => {

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/ingatlanlistazas"
    );

    const adatok = await valasz.json();

    setIngatlanok(adatok);

  } catch (err) {

    console.error(err);

  }

};

useEffect(() => {

  lekeres();

}, []);

const szerkesztes = (ingatlan) => {

  setSzerkesztettIngatlan(ingatlan);

  setUjIngatlan({
    tulajdonosid: ingatlan.tulajdonosid,
    cim: ingatlan.cim,
    tipus: ingatlan.tipus,
    szobakszama: ingatlan.szobakszama,
    haviar: ingatlan.haviar,
    leiras: ingatlan.leiras
  });
};
```

```
        setUrlapLathato(true);

    };

    const torles = async (id) => {

        if (!window.confirm("Biztosan törli az ingatlant?")) {
            return;
        }

        try {

            const valasz = await fetch(
                `http://localhost:3001/api/inglantorles/${id}`,
                {
                    method: "DELETE"
                }
            );

            const adat = await valasz.json();

            alert(adat.uzenet);

            lekeres();

        } catch (err) {

            console.error(err);

        }

    };

    const letrehozas = async (e) => {

        e.preventDefault();

        try {

            if (szerkesztettIngatlan) {

                const valasz = await fetch(
                    `http://localhost:3001/api/ingatlanmodositas/${szerkesztettIngatlan.id}`,
                    {
                        method: "PUT",
                        headers: {
```

```
        "Content-Type":
            "application/json"
    },
    body: JSON.stringify(
        ujIngatlan
    )
}
);

const adat =
    await valasz.json();

alert(adat.uzenet);

setSzerkesztettIngatlan(null);

setUrlapLathato(false);

lekeres();

return;
}

const valasz = await fetch(
    "http://localhost:3001/api/ingatlanletrehozas",
    {
        method: "POST",
        headers: {
            "Content-Type":
                "application/json"
        },
        body: JSON.stringify(
            ujIngatlan
        )
    }
);

const adat = await valasz.json();

alert(adat.uzenet);

setUjIngatlan({
    tulajdonosid: "",
    cim: "",
    tipus: "",
    szobakszama: "",
    haviar: "",
    leiras: ""
```

```
    });

    setUrlapLathato(false);

    lekeres();

  } catch (err) {

    console.error(err);

    alert("Hiba történt!");

  }

};

return (

  <div className="ingatlan-container">

    <h1>Ingatlanok</h1>

    <button
      className="uj-btn"
      onClick={() =>
        setUrlapLathato(
          !urlapLathato
        )
      }
    >
      {
        urlapLathato
          ? "Űrlap bezárása"
          : "Új ingatlan felvétele"
      }
    </button>

    {urlapLathato && (

      <form
        className="ingatlan-form"
        onSubmit={letrehozas}
      >

        <input
          type="number"
          placeholder="Tulajdonos ID"
          value={
```

```
        ujIngatlan.tulajdonosid
      }
      onChange={(e) =>
        setUjIngatlan({
          ...ujIngatlan,
          tulajdonosid:
            e.target.value
        })
      }
    />

    <input
      type="text"
      placeholder="Cím"
      value={ujIngatlan.cim}
      onChange={(e) =>
        setUjIngatlan({
          ...ujIngatlan,
          cim: e.target.value
        })
      }
    />

    <input
      type="text"
      placeholder="Típus"
      value={ujIngatlan.tipus}
      onChange={(e) =>
        setUjIngatlan({
          ...ujIngatlan,
          tipus:
            e.target.value
        })
      }
    />

    <input
      type="number"
      placeholder="Szobák száma"
      value={
        ujIngatlan.szobakszama
      }
      onChange={(e) =>
        setUjIngatlan({
          ...ujIngatlan,
          szobakszama:
            e.target.value
        })
      }
    />
```

```
    }  
  />  
  
  <input  
    type="number"  
    placeholder="Havi ár"  
    value={  
      ujIngatlan.haviar  
    }  
    onChange={(e) =>  
      setUjIngatlan({  
        ...ujIngatlan,  
        haviar:  
          e.target.value  
      })  
    }  
  />  
  
  <textarea  
    placeholder="Leírás"  
    value={  
      ujIngatlan.leiras  
    }  
    onChange={(e) =>  
      setUjIngatlan({  
        ...ujIngatlan,  
        leiras:  
          e.target.value  
      })  
    }  
  />  
  
  <button type="submit">  
  
    {  
      szerkesztettIngatlan  
        ? "Módosítás mentése"  
        : "Új ingatlan felvétele"  
    }  
  
  </button>  
  
</form>  
  
  )}  
  
  <div className="ingatlan-grid">
```

```
{ingatlanok.map((ingatlan) => (  
  
  <div  
    className="ingatlan-card"  
    key={ingatlan.id}  
  >  
  
    <h3>  
      {ingatlan.cim}  
    </h3>  
  
    <p>  
      <strong>  
        Típus:  
      </strong>{" "  
      {ingatlan.tipus}  
    </p>  
  
    <p>  
      <strong>  
        Szobák száma:  
      </strong>{" "  
      {ingatlan.szobakszama}  
    </p>  
  
    <p>  
      <strong>  
        Havi ár:  
      </strong>{" "  
      {ingatlan.haviar}  
      {" "}Ft  
    </p>  
  
    <p>  
      <strong>  
        Leírás:  
      </strong>{" "  
      {ingatlan.leiras}  
    </p>  
  
    <div className="gombok">  
  
      <button  
        className="modositas-btn"  
        onClick={() =>  
          szerkesztes(  
            ingatlan  
          )  
        >
```



```

        }
      >
        Módosítás
      </button>

      <button
        className="torles-btn"
        onClick={() =>
          torles(
            ingatlan.id
          )
        }
      >
        Törlés
      </button>

    </div>

  </div>

  )})

</div>

</div>

);
}

export default Ingatlanok;

```

1. Részlet az *Ingatlan.jsx* fájlból.

ingatlanok.css

Az ingatlanok.css fájlban elkészítettem az Ingatlanok oldal teljes megjelenését és formázását. Kialakítottam a kártyás nézetet, amely áttekinthető módon jeleníti meg az ingatlanok adatait. Stílust adtam az űrlapoknak, beviteli mezőknek és gomboknak annak érdekében, hogy a felület felhasználóbarát legyen. A módosítás és törlés gombok külön színeket kaptak, így a funkciók könnyen megkülönböztethetők.

```

.ingatlan-container {
  max-width: 1300px;
  margin: 0 auto;
  padding: 20px;
}

.ingatlan-container h1 {
  text-align: center;

```

```
    margin-bottom: 15px;
}

.uj-btn {
    display: block;
    margin: 0 auto 15px auto;
    background-color: #007bff;
    color: white;
    border: none;
    padding: 10px 20px;
    border-radius: 6px;
    font-weight: bold;
    cursor: pointer;
}

.ingatlan-form {
    background: white;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
    box-shadow: 0 0 8px rgba(0,0,0,0.1);
}

.ingatlan-form input,
.ingatlan-form textarea {
    width: 100%;
    padding: 10px;
    margin-bottom: 8px;
    border: 1px solid #ccc;
    border-radius: 5px;
    box-sizing: border-box;
}

.ingatlan-form textarea {
    min-height: 70px;
    resize: vertical;
}

.ingatlan-form button {
    width: 100%;
    background: #28a745;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 6px;
    cursor: pointer;
    font-weight: bold;
}
```

```
.ingatlan-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
  gap: 12px;
}

.ingatlan-card {
  background: white;
  border-radius: 8px;
  padding: 15px;
  box-shadow: 0 0 8px rgba(0,0,0,0.1);
}

.ingatlan-card h3 {
  margin: 0 0 10px 0;
  color: #007bff;
}

.ingatlan-card p {
  margin: 4px 0;
}

.gombok {
  display: flex;
  gap: 8px;
  margin-top: 12px;
}

.modositas-btn,
.torles-btn {
  flex: 1;
  border: none;
  padding: 8px;
  border-radius: 6px;
  font-weight: bold;
  cursor: pointer;
}

.modositas-btn {
  background: orange;
  color: white;
}

.torles-btn {
  background: #dc3545;
  color: white;
}
```

2. Részlet az ingatlan.css fájlból.

localhost:3000/ingatlanok

Ingatlankezelő rendszer

Ingatlanok Bérlok Szerződések Fizetések Profil Kijelentkezés

Ingatlanok

Úrlap bezárása

Tulajdonos ID

Cím

Típus

Szobák száma

Havi ár

Leírás

Új ingatlan felvétele

3. Az ingatlan felvétel HTML űrlapra.

localhost:3000/ingatlanok

Ingatlankezelő rendszer

Ingatlanok Bérlok Szerződések Fizetések Profil Kijelentkezés

Ingatlanok

Úrlap bezárása

1

Budapest, Petőfi utca 8.

Társasház

3

180000.00

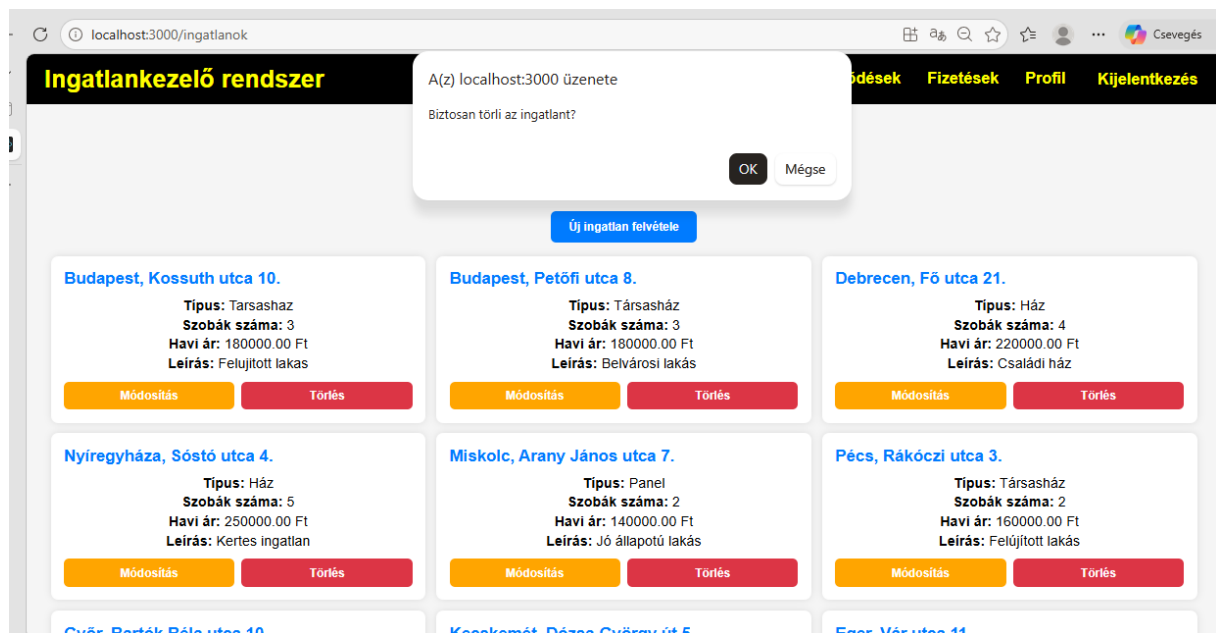
Belvárosi lakás

Módosítás mentése

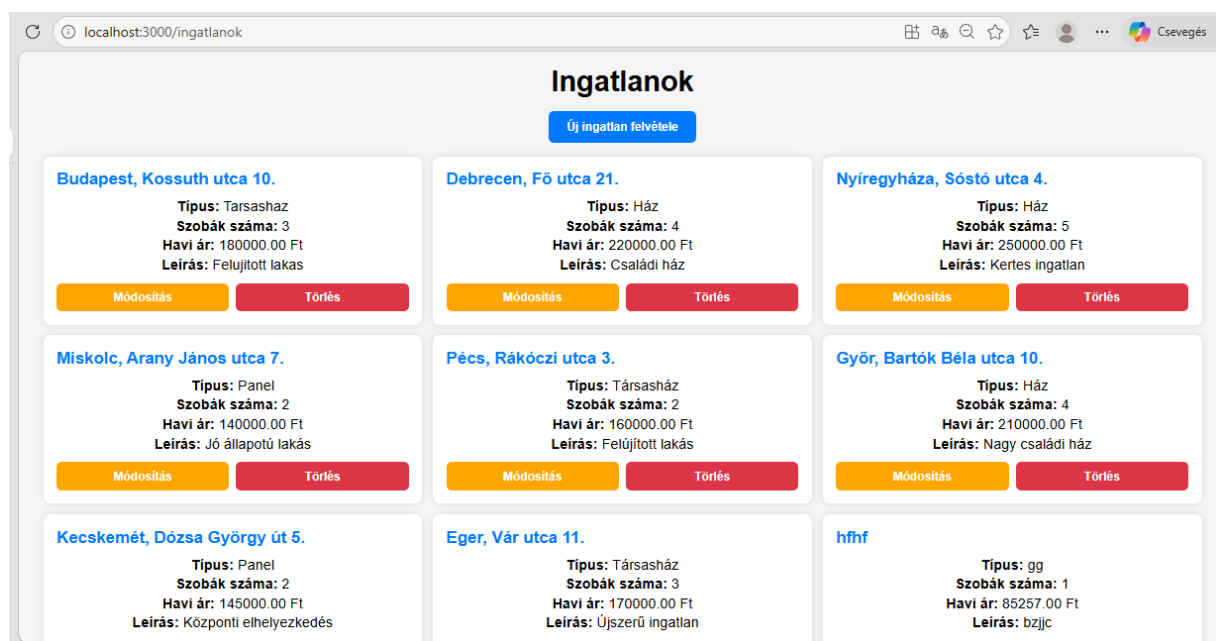
4. Részlet az ingatlan módosítás HTML űrlapból.

Szakmai Gyakorlat II. – Frontend – Backend integrációs szakasz

Orosz Kristóf – EYZWG9



5. Részlet az ingatlan törlés funkciójáról.



6. Részlet az ingatlan HTML oldalról.

ingatlanlistazas.js

Az ingatlanlistazas.js fájlban elkészítettem az ingatlanok listázására szolgáló backend végpontot. A végpont adatbázis-kapcsolatot létesít, majd lekérdezi az ingatlanok tábla összes rekordját. A lekérdezett adatokat JSON formátumban küldi vissza a frontend számára. Gondoskodtam a hibakezelésről, hogy adatbázis-hiba esetén megfelelő válasz érkezzon.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");
```

```
router.get("/", async (req, res) => {  
  
  let conn;  
  
  try {  
  
    conn = await pool.getConnection();  
  
    const adatok = await conn.query(  
      "SELECT * FROM ingatlanok"  
    );  
  
    res.json(adatok);  
  
  } catch (err) {  
  
    res.status(500).json(err);  
  
  } finally {  
  
    if(conn) conn.release();  
  
  }  
  
});  
  
module.exports = router;
```

7. Részlet az ingatlanlistazas.js fájlból.

ingatlanletrehozás.js

Az ingatlanletrehozás.js fájlban megvalósítottam az új ingatlanok létrehozását végző backend funkciót. A végpont fogadja a frontend által küldött adatokat, például a címet, típust, szobaszámot és havi bérleti díjat. Az adatokat SQL INSERT utasítással rögzíti az adatbázis ingatlanok táblájába. Sikeres mentés esetén a rendszer visszajelző üzenetet küld a frontend számára.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.post("/", async (req, res) => {  
  
  const {  
    tulajdonosid,  
    cim,  
    tipus,  
    szobakszama,
```

```
        haviar,  
        leiras  
    } = req.body;  
  
    let conn;  
  
    try {  
  
        conn = await pool.getConnection();  
  
        await conn.query(  
            `INSERT INTO ingatlanok  
            (tulajdonosid, cim, tipus, szobakszama, haviar, leiras)  
            VALUES (?, ?, ?, ?, ?, ?)` ,  
            [  
                tulajdonosid,  
                cim,  
                tipus,  
                szobakszama,  
                haviar,  
                leiras  
            ]  
        );  
  
        res.json({  
            siker: true,  
            uzenet: "Ingatlan sikeresen létrehozva!"  
        });  
  
    } catch (err) {  
  
        res.status(500).json(err);  
  
    } finally {  
  
        if(conn) conn.release();  
  
    }  
});  
  
module.exports = router;
```

8. Részlet az ingatlanletrehozás.js fájlból.

ingatlanmodositas.js

Az ingatlanmodositas.js fájlban elkészítettem az ingatlanok módosítására szolgáló backend végpontot. A rendszer az URL-ben kapott azonosító alapján kiválasztja a módosítani kívánt rekordot. A frontendről

érkező új adatokat SQL UPDATE utasítással menti az adatbázisba. A művelet sikeres végrehajtása után visszajelző üzenetet küld a kliens számára.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

  const {
    cim,
    tipus,
    szobakszama,
    haviar,
    leiras
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `UPDATE ingatlanok
      SET
      cim = ?,
      tipus = ?,
      szobakszama = ?,
      haviar = ?,
      leiras = ?
      WHERE id = ?`,
      [
        cim,
        tipus,
        szobakszama,
        haviar,
        leiras,
        req.params.id
      ]
    );

    res.json({
      siker: true,
      uzenet: "Ingatlan módosítva!"
    });

  } catch (err) {
```



```
        res.status(500).json(err);

    } finally {

        if(conn) conn.release();

    }

});

module.exports = router;
```

9. Részlet az ingatlanmodositas.js fájlból.

ingatlantorles.js

Az ingatlantorles.js fájlban kialakítottam az ingatlanok törléséhez szükséges backend végpontot. A rendszer az URL-ben kapott azonosító alapján határozza meg a törlendő rekordot. A törlést SQL DELETE utasítással hajtja végre az adatbázisban. Sikeres végrehajtás esetén a frontend tájékoztató üzenetet kap a művelet eredményéről.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

    let conn;

    try {

        conn = await pool.getConnection();

        await conn.query(
            "DELETE FROM ingatlanok WHERE id = ?",
            [req.params.id]
        );

        res.json({
            siker: true,
            uzenet: "Ingatlan törölve!"
        });

    } catch (err) {

        res.status(500).json(err);

    } finally {
```

```
        if(conn) conn.release();

    }

});

module.exports = router;
```

10. Részlet az ingatlanlistazas.js fájlból.

Bérlők oldalak és funkciók beállítása

Berlok.jsx

A Berlok.jsx fájlban elkészítettem a bérlők adatainak megjelenítésére szolgáló React komponenst. Az oldal betöltésekor lekérem a backendből a bérlők adatait, majd eltárolom azokat a komponens állapotában. A kapott adatokat kártyás elrendezésben jelenítem meg, így a felhasználók könnyen áttekinthetik a bérlők adatait. Minden kártyán megjelenítem a bérlő nevét, e-mail címét, telefonszámát, személyi számát és lakcímét.

```
import { useEffect, useState } from "react";
import "../styles/berlok.css";

function Berlok() {

  const [berlok, setBerlok] = useState([]);

  const berlokBetoltese = async () => {

    try {

      const response = await fetch(
        "http://localhost:3001/api/berlolistazas"
      );

      const data = await response.json();

      setBerlok(
        Array.isArray(data)
          ? data
          : []
      );

    } catch (error) {

      console.error(error);

      alert(
        "Nem sikerült betölteni a bérlőket!"
      );
    }
  };
}
```

```
    );  
  
  }  
};  
  
useEffect(() => {  
  berlokBetoltese();  
}, []);  
  
return (  
  <div className="tenant-container">  
    <h1>Bérlők</h1>  
  
    <div className="tenant-grid">  
      {berlok.map((berlo) => (  
        <div  
          key={berlo.id}  
          className="tenant-card"  
        >  
          <h3>  
            {berlo.nev}  
          </h3>  
  
          <p>  
            <strong>  
              E-mail:  
            </strong>{" "  
            {  
              berlo.email ||  
                "Nincs megadva"  
            }  
          </p>  
  
          <p>  
            <strong>  
              Telefonszám:  
            </strong>{" "  
            {  
              berlo.telefonszam ||  
                "Nincs megadva"
```

44

```
padding: 20px;
}

.tenant-container h1 {
  text-align: center;
  margin-bottom: 20px;
}

.add-btn {
  display: block;
  margin: 0 auto 20px auto;
  padding: 12px 20px;
  background-color: #007bff;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-weight: bold;
}

.add-btn:hover {
  background-color: #0056b3;
}

.tenant-form {
  background: white;
  padding: 20px;
  border-radius: 10px;
  margin-bottom: 25px;
  box-shadow: 0 0 15px rgba(0,0,0,0.15);
}

.tenant-form input,
.tenant-form textarea {
  width: 100%;
  padding: 12px;
  margin-bottom: 10px;
  border: 1px solid #ccc;
  border-radius: 6px;
  box-sizing: border-box;
}

.tenant-form button {
  width: 100%;
  padding: 12px;
  background: #28a745;
  color: white;
  border: none;
```

```
    border-radius: 6px;
    font-weight: bold;
    cursor: pointer;
}

.tenant-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(380px, 1fr));
    gap: 20px;
}

.tenant-card {
    background: white;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 0 15px rgba(0,0,0,0.12);
    transition: 0.3s;
}

.tenant-card:hover {
    transform: translateY(-5px);
}

.tenant-card h3 {
    text-align: center;
    color: #007bff;
    margin-bottom: 15px;
}

.tenant-card p {
    margin: 8px 0;
    line-height: 1.5;
}

.buttons {
    display: flex;
    gap: 10px;
    margin-top: 15px;
}

.edit-btn {
    flex: 1;
    background: orange;
    color: white;
    border: none;
    padding: 10px;
    border-radius: 6px;
    font-weight: bold;
}
```

```
        cursor: pointer;
    }

    .delete-btn {
        flex: 1;
        background: #dc3545;
        color: white;
        border: none;
        padding: 10px;
        border-radius: 6px;
        font-weight: bold;
        cursor: pointer;
    }

    .assign-btn {
        flex: 1;
        background: #28a745;
        color: white;
        border: none;
        padding: 10px;
        border-radius: 6px;
        font-weight: bold;
        cursor: pointer;
    }
}
```

berlolistazas.js

A berlolistazas.js fájlban létrehoztam a bérlők adatainak lekérdezésére szolgáló backend végpontot. A végpont kapcsolatot létesít a MariaDB adatbázissal, majd lekérdezi a users tábla tartalmát. Az eredményül kapott adatokat JSON formátumban küldöm vissza a frontend számára. A hibák kezelésére try-catch szerkezetet alkalmaztam, amely naplózza az esetleges problémákat. A kapcsolat lezárását minden esetben biztosítom, hogy elkerüljem az adatbázis-kapcsolatok felesleges foglaltságát.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

    let conn;

    try {

        conn = await pool.getConnection();

        const adatok = await conn.query(
            "SELECT * FROM users"
        );

    }
```

```
        res.json(adatok);

    } catch (err) {

        console.error(err);
        res.status(500).json(err);

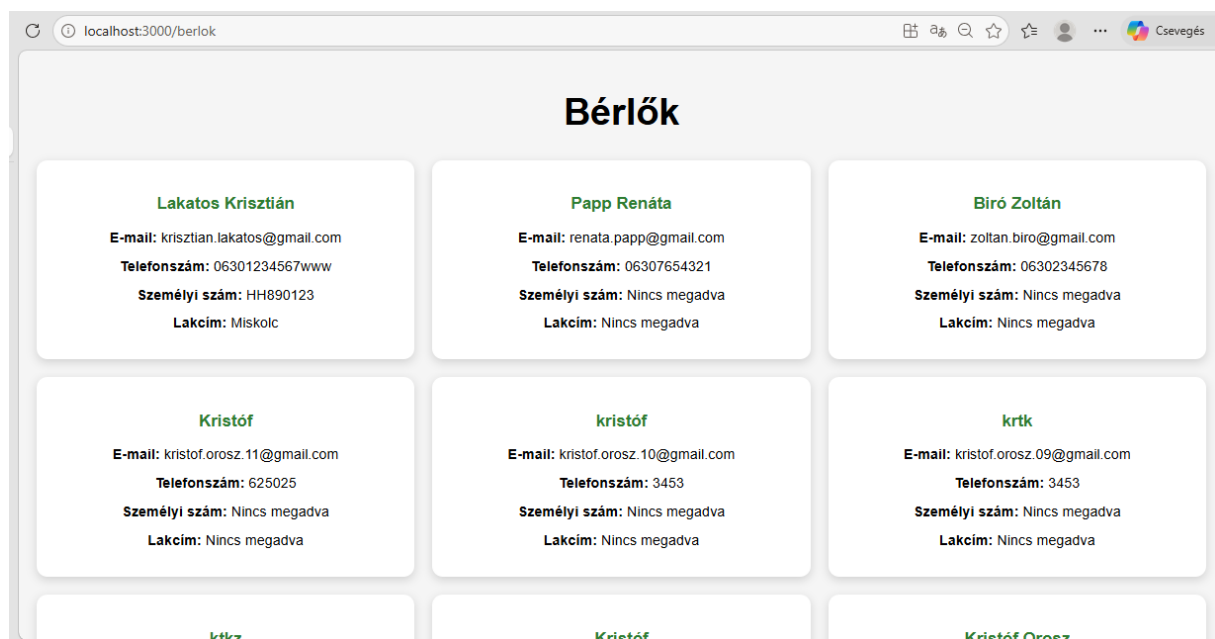
    } finally {

        if (conn) conn.release();

    }

});

module.exports = router;
```



1.Kép: Bérlők HTML oldala.

Szerződések funkciók beállítása

Szerződések kezelése

A szerződések kezelése funkció fejlesztése során elkészítettem a szerződések nyilvántartására szolgáló felhasználói felületet. Az űrlapon lehetővé tettem új szerződések létrehozását, ahol kiválasztható az ingatlan és a bérlő, valamint megadhatók a szerződéshez tartozó adatok. A rendszer automatikusan betölti az adatbázisban található ingatlanokat és bérlőket, így a felhasználónak nem kell manuálisan megadnia az azonosítókat. Megvalósítottam a szerződések módosításának lehetőségét is, amely során a kiválasztott szerződés adatai automatikusan betöltődnek az űrlapba szerkesztés céljából. A funkció tesztelése során ellenőriztem az új szerződések létrehozását, a meglévő adatok módosítását, valamint a szerződések törlését is. A rendszerben kialakítottam a szerződések státusz szerinti szűrésének

lehetőségét. A felhasználó kiválaszthatja, hogy az összes, az aktív vagy a lejárt szerződéseket szeretné megjeleníteni. A kiválasztott szűrő alapján a frontend dinamikusan szűri a megjelenített adatokat. Ezzel jelentősen egyszerűbbé vált a nagyobb mennyiségű szerződés áttekintése. A funkció működését különböző státuszú tesztadatokkal ellenőriztem.

```
import { useEffect, useState } from "react";
import "../styles/szerzodesek.css";

function Szerzodesek() {

  const [szerzodesek, setSzerzodesek] = useState([]);
  const [berlok, setBerlok] = useState([]);
  const [ingatlanok, setIngatlanok] = useState([]);
  const [szerkesztettId, setSzerkesztettId] = useState(null);
  const [statuszSzuro, setStatuszSzuro] = useState("Mind");

  const [ujSzerzodes, setUjSzerzodes] = useState({
    ingatlansid: "",
    berloidsid: "",
    kezdetdatum: "",
    vegdatum: "",
    havidij: "",
    statusz: "Aktív"
  });

  const szerzodesekBetoltese = async () => {

    try {

      const valasz = await fetch(
        "http://localhost:3001/api/szerzodeslistazas"
      );

      const adatok = await valasz.json();

      setSzerzodesek(adatok);

    } catch (err) {

      console.error(err);

    }

  };

  const berlokBetoltese = async () => {

    try {
```

```
        const valasz = await fetch(
            "http://localhost:3001/api/berlolistazas"
        );

        const adatok = await valasz.json();

        setBerlok(adatok);

    } catch (err) {

        console.error(err);

    }
};

const ingatlanokBetoltese = async () => {

    try {

        const valasz = await fetch(
            "http://localhost:3001/api/ingatlanlistazas"
        );

        const adatok = await valasz.json();

        setIngatlanok(adatok);

    } catch (err) {

        console.error(err);

    }
};

useEffect(() => {

    szerzodesekBetoltese();
    berlokBetoltese();
    ingatlanokBetoltese();

}, []);

const mentes = async (e) => {

    e.preventDefault();
```

```
try {

    let url =
        "http://localhost:3001/api/szerzodesletrehozas";

    let method = "POST";

    if (szerkesztettId) {

        url =
            `http://localhost:3001/api/szerzodesmodositas/${szerkesztettId}`;

        method = "PUT";

    }

    const valasz = await fetch(
        url,
        {
            method,
            headers: {
                "Content-Type":
                    "application/json"
            },
            body: JSON.stringify(
                ujSzerzodes
            )
        }
    );

    const adat =
        await valasz.json();

    alert(adat.uzenet);

    setUjSzerzodes({
        ingatlanid: "",
        berloid: "",
        kezdetdatum: "",
        vegdatum: "",
        havidij: "",
        statusz: "Aktív"
    });

    setSzerkesztettId(null);

    szerzodesekBetoltese();
```

```
    } catch (err) {

        console.error(err);

        alert("Hiba történt!");

    }

};

const torles = async (id) => {

    if (
        !window.confirm(
            "Biztosan törölnöd a szerződést?"
        )
    ) {
        return;
    }

    try {

        const valasz = await fetch(
            `http://localhost:3001/api/szerzodestorles/${id}`,
            {
                method: "DELETE"
            }
        );

        const adat =
            await valasz.json();

        alert(adat.uzenet);

        szerzodesekBetoltese();

    } catch (err) {

        console.error(err);

    }

};

return (

    <div className="tenant-container">
```

```
<h1>Szerződések</h1>

<form
  className="tenant-form"
  onSubmit={mentes}
>

  <select
    value={
      ujSzerzodes.ingatlanid
    }
    onChange={(e) =>
      setUjSzerzodes({
        ...ujSzerzodes,
        ingatlanid:
          e.target.value
      })
    }
    required
  >

    <option value="">
      Válassz ingatlant
    </option>

    {ingatlanok.map(
      (ingatlan) => (

        <option
          key={
            ingatlan.id
          }
          value={
            ingatlan.id
          }
        >
          {
            ingatlan.cim
          }
        </option>

      )
    )}

  </select>

  <select
    value={
```

```
        ujSzerzodes.berloid
      }
      onChange={(e) =>
        setUjSzerzodes({
          ...ujSzerzodes,
          berloid:
            e.target.value
        })
      }
    }
    required
  >

  <option value="">
    Válassz bérlőt
  </option>

  {berlok.map(
    (berlo) => (

      <option
        key={
          berlo.id
        }
        value={
          berlo.id
        }
      >
        {
          berlo.nev
        }
      </option>

    )
  )}

</select>

<input
  type="date"
  value={
    ujSzerzodes.kezdetdatum
  }
  onChange={(e) =>
    setUjSzerzodes({
      ...ujSzerzodes,
      kezdetdatum:
        e.target.value
    })
  }
/>
```

```
    }
    required
  />

  <input
    type="date"
    value={
      ujSzerzodes.vegdatum
    }
    onChange={(e) =>
      setUjSzerzodes({
        ...ujSzerzodes,
        vegdatum:
          e.target.value
      })
    }
    required
  />

  <input
    type="number"
    placeholder="Havi díj"
    value={
      ujSzerzodes.havidij
    }
    onChange={(e) =>
      setUjSzerzodes({
        ...ujSzerzodes,
        havidij:
          e.target.value
      })
    }
    required
  />

  <select
    value={
      ujSzerzodes.statusz
    }
    onChange={(e) =>
      setUjSzerzodes({
        ...ujSzerzodes,
        statusz:
          e.target.value
      })
    }
  >
```

```
        <option value="Aktív">
            Aktív
        </option>

        <option value="Lejárt">
            Lejárt
        </option>

    </select>

    <button
        type="submit"
    >
        {
            szerkesztettId
            ? "Módosítás mentése"
            : "Szerződés létrehozása"
        }
    </button>

</form>

<div className="filter-container">

    <select
        value={statuszSzuro}
        onChange={(e) =>
            setStatuszSzuro(
                e.target.value
            )
        }
    >

        <option value="Mind">
            Összes szerződés
        </option>

        <option value="Aktív">
            Aktív
        </option>

        <option value="Lejárt">
            Lejárt
        </option>

    </select>

</div>
```



```
<div className="tenant-grid">

  {szerzodesek
    .filter((szerzodes) => {

      if (
        statuszSzuro ===
        "Mind"
      ) {
        return true;
      }

      return (
        szerzodes.statusz ===
        statuszSzuro
      );
    })
    .map(
      (szerzodes) => (

        <div
          className="tenant-card"
          key={
            szerzodes.id
          }
        >

          <h3>
            {
              szerzodes.ingatlancim
            }
          </h3>

          <p>
            <strong>
              Bérző:
            </strong>{" "}
            {
              szerzodes.berlonev
            }
          </p>

          <p>
            <strong>
              Kezdet:
            </strong>{" "}
```

```
        {
            new Date(
                szerzodes.kezdetdatum
            ).toLocaleDateString(
                "hu-HU"
            )
        }
    </p>

    <p>
        <strong>
            Vége:
        </strong>{" "}
        {
            new Date(
                szerzodes.vegdatum
            ).toLocaleDateString(
                "hu-HU"
            )
        }
    </p>

    <p>
        <strong>
            Havi díj:
        </strong>{" "}
        {
            szerzodes.havidij
        } Ft
    </p>

    <p>

        <strong>
            Státusz:
        </strong>

        <span
            className={
                szerzodes.statusz ===
                "Aktív"
                ? "active-status"
                : "expired-status"
            }
        >
            {" "}
            {
                szerzodes.statusz
```

```
    }  
  </span>  
  
</p>  
  
<div className="buttons">  
  
  <button  
    type="button"  
    className="edit-btn"  
    onClick={() => {  
  
      setSzerkesztettId(  
        szerzodes.id  
      );  
  
      setUjSzerzodes({  
        ingatlanid:  
          szerzodes.ingatlanid,  
        berloid:  
          szerzodes.berloid,  
        kezdetdatum:  
          szerzodes.kezdetdatum?.split("T")[0] ||  
          szerzodes.kezdetdatum,  
        vegdatum:  
          szerzodes.vegdatum?.split("T")[0] ||  
          szerzodes.vegdatum,  
        havidiij:  
          szerzodes.havidiij,  
        statusz:  
          szerzodes.statusz  
      });  
  
      window.scrollTo({  
        top: 0,  
        behavior: "smooth"  
      });  
  
    }}  
  >  
    Módosítás  
  </button>  
  
  <button  
    type="button"  
    className="delete-btn"  
    onClick={() =>  
      torles(  

```

```
                szerzodes.id
            )
        }
    >
    Törlés
</button>

</div>

</div>

    )
  })

</div>

</div>

);

}

export default Szerzodesek;
```

Szerződések oldal stíluslapja

A szerződések oldal megjelenítéséhez külön CSS stíluslapot készítettem. A célom egy letisztult, könnyen áttekinthető és felhasználóbarát felület kialakítása volt. A szerződések kártyás elrendezésben jelennek meg, ami megkönnyíti az adatok áttekintését. Külön stílusokat alkalmaztam az űrlapokra, a gombokra és a státuszjelölésekre is. A reszponzív kialakításnak köszönhetően az oldal kisebb képernyőkön és mobil eszközökön is megfelelően használható.

```
.tenant-container {
  max-width: 1400px;
  margin: 30px auto;
  padding: 20px;
}

.tenant-container h1 {
  text-align: center;
  margin-bottom: 30px;
  font-size: 42px;
}

.tenant-form {
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
```

```
    background: white;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 3px 10px rgba(0,0,0,0.15);
    margin-bottom: 30px;
}

.tenant-form select,
.tenant-form input {
    flex: 1;
    min-width: 180px;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 8px;
}

.tenant-form button {
    background: #2e7d32;
    color: white;
    border: none;
    padding: 12px 25px;
    border-radius: 8px;
    cursor: pointer;
}

.tenant-form button:hover {
    background: #1b5e20;
}

.tenant-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill,minmax(320px,1fr));
    gap: 20px;
}

.tenant-card {
    background: white;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 3px 10px rgba(0,0,0,0.15);
}

.tenant-card h3 {
    margin-bottom: 15px;
    color: #2e7d32;
}

.tenant-card p {
```

```
    margin: 8px 0;
}

.buttons {
    margin-top: 15px;
    display: flex;
    gap: 10px;
}

.edit-btn {
    background: #1976d2;
    color: white;
    border: none;
    padding: 10px 15px;
    border-radius: 8px;
    cursor: pointer;
}

.edit-btn:hover {
    background: #0d47a1;
}

.delete-btn {
    background: #d32f2f;
    color: white;
    border: none;
    padding: 10px 15px;
    border-radius: 8px;
    cursor: pointer;
}

.delete-btn:hover {
    background: #b71c1c;
}

.filter-container {
    margin: 20px 0;
}

.filter-container select {
    padding: 10px;
    border-radius: 8px;
}

.active-status {
    color: green;
    font-weight: bold;
}
```

```
.expired-status {  
  color: red;  
  font-weight: bold;  
}
```

Szerződés létrehozása

A szerződés létrehozása funkció elkészítése során kialakítottam az adatbeviteli űrlapot, amely lehetővé teszi új szerződések rögzítését. Az űrlapon kiválasztható az ingatlan és a bérlő, valamint megadható a szerződés kezdő és befejező dátuma, a havi díj és a státusz. A megadott adatokat POST kérés segítségével továbbítottam a backend felé. A backend az adatokat ellenőrizte, majd eltárolta a szerzodesek adatbázistáblában. A funkció tesztelése során ellenőriztem, hogy az új szerződések sikeresen bekerülnek az adatbázisba.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.post("/", async (req, res) => {  
  
  const {  
    ingatlanid,  
    berloid,  
    kezdetdatum,  
    vegdatum,  
    havidij,  
    statusz  
  } = req.body;  
  
  let conn;  
  
  try {  
  
    conn = await pool.getConnection();  
  
    await conn.query(  
      `INSERT INTO szerzodesek  
      (ingatlanid, berloid, kezdetdatum, vegdatum, havidij, statusz)  
      VALUES (?, ?, ?, ?, ?, ?)`,  
      [  
        ingatlanid,  
        berloid,  
        kezdetdatum,  
        vegdatum,  
        havidij,  
        statusz  
      ]  
    );  
  }  
}
```

```
        ]
    );

    res.json({
        siker: true,
        uzenet: "Szerződés sikeresen létrehozva!"
    });

} catch (err) {

    res.status(500).json(err);

} finally {

    if(conn) conn.release();

}

});

module.exports = router;
```

Szerződések listázása

A szerződések listázása funkció megvalósításával lehetővé tettem a rendszerben tárolt szerződések megjelenítését. A frontend GET kérés segítségével lekérdezi a backendtől a szerződések adatait. A megjelenített listában látható az ingatlan címe, a bérlő neve, a szerződés időtartama, a havi díj és a státusz. A felhasználói felület áttekinthető kártyás megjelenítést kapott. A funkció működését több tesztadattal is ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

    let conn;

    try {

        conn = await pool.getConnection();

        const adatok = await conn.query(`
            SELECT
                szerzodesek.*,
                users.nev AS berlonev,
                ingatlanok.cim AS ingatlancim
            FROM szerzodesek
            INNER JOIN users
```



```
        ON users.id = szerzodesek.berloid
        INNER JOIN ingatlanok
        ON ingatlanok.id = szerzodesek.ingatlanid
        ORDER BY szerzodesek.id DESC
    `);

    res.json(adatok);

} catch (err) {

    console.error(err);

    res.status(500).json(err);

} finally {

    if (conn) {
        conn.release();
    }

}

});

module.exports = router;
```

Szerződés módosítása

A szerződés módosítása funkció elkészítése során lehetőséget biztosítottam a meglévő szerződések adatainak szerkesztésére. A kiválasztott szerződés adatai automatikusan betöltődnek az űrlapba. A felhasználó módosíthatja a kezdő és végdátumot, a havi díjat, valamint a szerződés státuszát. A mentést követően a rendszer PUT kérés segítségével frissíti az adatbázisban tárolt adatokat. A módosítások helyességét sikeres adatbázis-frissítéssel ellenőriztem.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.put("/:id", async (req, res) => {

    const {
        kezdetdatum,
        vegdatum,
        havidij,
        statusz
    } = req.body;
```

```
let conn;

try {

  conn = await pool.getConnection();

  await conn.query(
    `UPDATE szerzodesek
    SET
    kezdetdatum = ?,
    vegdatum = ?,
    havidij = ?,
    statusz = ?
    WHERE id = ?`,
    [
      kezdetdatum,
      vegdatum,
      havidij,
      statusz,
      req.params.id
    ]
  );

  res.json({
    siker: true,
    uzenet: "Szerzodes modositva!"
  });

} catch (err) {

  res.status(500).json(err);

} finally {

  if(conn) conn.release();

}

});

module.exports = router;
```

Szerződés törlése

A szerződés törlése funkció megvalósításával lehetővé tettem a már nem szükséges szerződések eltávolítását. A törlés előtt a rendszer megerősítést kér a felhasználótól a véletlen adatvesztés elkerülése érdekében. Jóváhagyás esetén DELETE kérés kerül elküldésre a backendnek. A backend

eltávolítja a kiválasztott rekordot az adatbázisból. A funkció tesztelése során ellenőriztem, hogy a törölt szerződések a listából és az adatbázisból is megfelelően eltűnnek.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.delete("/:id", async (req, res) => {

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      "DELETE FROM szerzodesek WHERE id = ?",
      [req.params.id]
    );

    res.json({
      siker: true,
      uzenet: "Szerzodes torolve!"
    });

  } catch (err) {

    res.status(500).json(err);

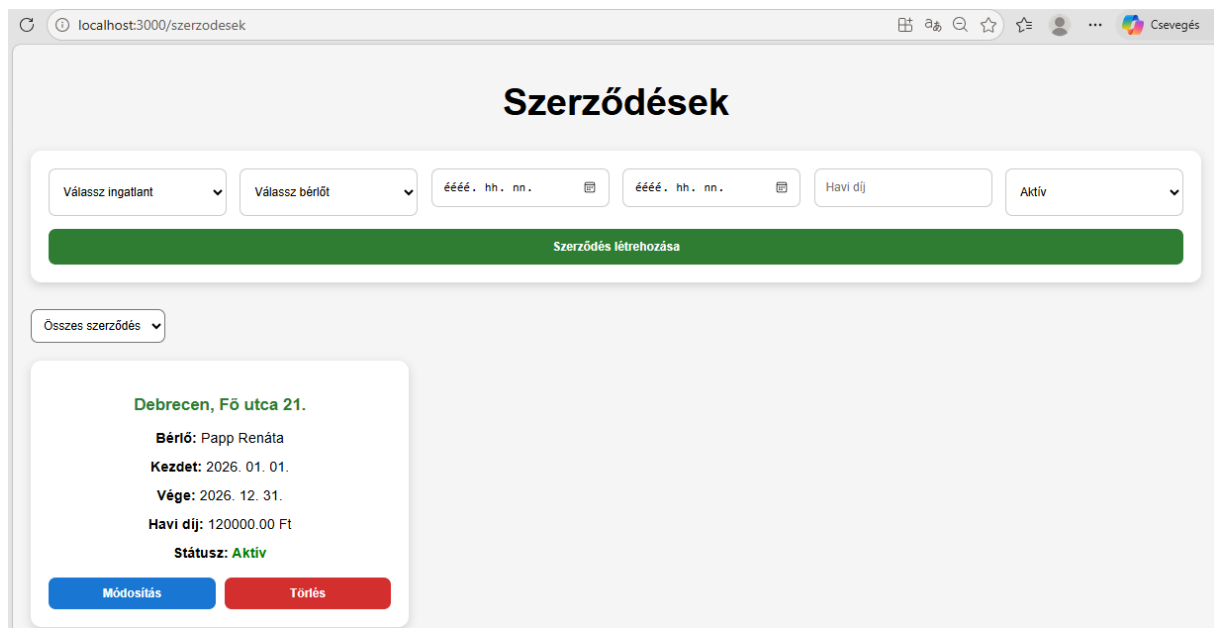
  } finally {

    if(conn) conn.release();

  }

});

module.exports = router;
```



1.Kép: Szerződések HTML oldal kimenete.

Fizetések létrehozása

1. Fizetesek.jsx

A fizetések kezeléséhez elkészítettem a Fizetesek.jsx komponenst. A komponensben React state-ek segítségével tároltam a fizetések, a szerződések és az űrlap adatait. Gondoskodtam arról, hogy az oldal betöltésekor automatikusan lekérdeződjenek a szükséges adatok a backendből. Az űrlap segítségével lehetővé tettem új fizetések rögzítését és a meglévő fizetések módosítását.

```
import { useEffect, useState } from "react";
import "../styles/fizetesek.css";

function Fizetesek() {

  const [fizetesek, setFizetesek] = useState([]);
  const [szerzodesek, setSzerzodesek] = useState([]);
  const [szerkesztettId, setSzerkesztettId] = useState(null);
  const [statuszSzuro, setStatuszSzuro] = useState("Mind");

  const [ujFizetes, setUjFizetes] = useState({
    szerzodesid: "",
    osszeg: "",
    fizetesdatum: "",
    statusz: "Teljesítve",
    fizetesmod: "",
    megjegyzes: ""
  });

  const fizetesekBetoltese = async () => {
```

```
    try {

      const valasz = await fetch(
        "http://localhost:3001/api/fizeteslistazas"
      );

      const adatok = await valasz.json();

      setFizetesek(adatok);

    } catch (err) {

      console.error(err);

    }

  };

const szerzodesekBetoltese = async () => {

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/szerzodeslistazas"
    );

    const adatok = await valasz.json();

    setSzerzodesek(adatok);

  } catch (err) {

    console.error(err);

  }

};

useEffect(() => {

  fizetesekBetoltese();
  szerzodesekBetoltese();

}, []);

const mentes = async (e) => {
```

```
e.preventDefault();

try {

    let url =
        "http://localhost:3001/api/fizetesletrehozas";

    let method = "POST";

    if (szerkesztettId) {

        url =
            `http://localhost:3001/api/fizetesmodositas/${szerkesztettId}`;

        method = "PUT";

    }

    const valasz = await fetch(
        url,
        {
            method,
            headers: {
                "Content-Type":
                    "application/json"
            },
            body: JSON.stringify(
                ujFizetes
            )
        }
    );

    const adat =
        await valasz.json();

    alert(adat.uzenet);

    setUjFizetes({
        szerzodesid: "",
        osszeg: "",
        fizetesdatum: "",
        statusz: "Teljesítve",
        fizetesmod: "",
        megjegyzes: ""
    });

    setSzerkesztettId(null);
```

```
        fizetesekBetoltese();

    } catch (err) {

        console.error(err);

        alert("Hiba történt!");

    }

};

const torles = async (id) => {

    if (
        !window.confirm(
            "Biztosan töröld a fizetést?"
        )
    ) {
        return;
    }

    try {

        const valasz = await fetch(
            `http://localhost:3001/api/fizetestorles/${id}`,
            {
                method: "DELETE"
            }
        );

        const adat =
            await valasz.json();

        alert(adat.uzenet);

        fizetesekBetoltese();

    } catch (err) {

        console.error(err);

    }

};

return (
```

```
<div className="tenant-container">

  <h1>Fizetések</h1>

  <form
    className="tenant-form"
    onSubmit={mentes}
  >

    <select
      value={
        ujFizetes.szerzodesid
      }
      onChange={(e) =>
        setUjFizetes({
          ...ujFizetes,
          szerzodesid:
            e.target.value
        })
      }
      required
    >

      <option value="">
        Válassz szerződést
      </option>

      {szerzodesek.map(
        (szerzodes) => (

          <option
            key={
              szerzodes.id
            }
            value={
              szerzodes.id
            }
          >
            {
              szerzodes.berlonev
            }
          </option>

        )
      )}

    </select>
```



```
<input
  type="number"
  placeholder="Összeg"
  value={
    ujFizetes.osszeg
  }
  onChange={(e) =>
    setUjFizetes({
      ...ujFizetes,
      osszeg:
        e.target.value
    })
  }
  required
/>

<input
  type="date"
  value={
    ujFizetes.fizetesdatum
  }
  onChange={(e) =>
    setUjFizetes({
      ...ujFizetes,
      fizetesdatum:
        e.target.value
    })
  }
  required
/>

<select
  value={
    ujFizetes.statusz
  }
  onChange={(e) =>
    setUjFizetes({
      ...ujFizetes,
      statusz:
        e.target.value
    })
  }
>

  <option value="Teljesítve">
    Teljesítve
  </option>
```

```
        <option value="Függőben">
            Függőben
        </option>

        <option value="Sikertelen">
            Sikertelen
        </option>

    </select>

    <input
        type="text"
        placeholder="Fizetési mód"
        value={
            ujFizetes.fizetesmod
        }
        onChange={ (e) =>
            setUjFizetes({
                ...ujFizetes,
                fizetesmod:
                    e.target.value
            })
        }
        required
    />

    <input
        type="text"
        placeholder="Megjegyzés"
        value={
            ujFizetes.megjegyzes
        }
        onChange={ (e) =>
            setUjFizetes({
                ...ujFizetes,
                megjegyzes:
                    e.target.value
            })
        }
    />

    <button
        type="submit"
    >
        {
            szerkesztettId
                ? "Módosítás mentése"
                : "Fizetés rögzítése"
```

```
    }  
  </button>  
  
</form>  
  
<div className="filter-container">  
  
  <select  
    value={statuszSzuro}  
    onChange={(e) =>  
      setStatuszSzuro(  
        e.target.value  
      )  
    }  
  >  
  
    <option value="Mind">  
      Összes fizetés  
    </option>  
  
    <option value="Teljesítve">  
      Teljesítve  
    </option>  
  
    <option value="Függőben">  
      Függőben  
    </option>  
  
    <option value="Sikertelen">  
      Sikertelen  
    </option>  
  
  </select>  
  
</div>  
  
<div className="tenant-grid">  
  
  {fizetesek  
    .filter((fizetes) => {  
  
      if (  
        statuszSzuro ===  
        "Mind"  
      ) {  
        return true;  
      }  
    })  
  }
```

```
        return (
            fizetes.statusz ===
            statuszSzuro
        );
    })
    .map(
        (fizetes) => (

            <div
                className="tenant-card"
                key={
                    fizetes.id
                }
            >

                <h3>
                    {
                        fizetes.berlonev
                    }
                </h3>

                <p>
                    <strong>
                        Összeg:
                    </strong>{" "}
                    {
                        fizetes.osszeg
                    } Ft
                </p>

                <p>
                    <strong>
                        Dátum:
                    </strong>{" "}
                    {
                        new Date(
                            fizetes.fizetesdatum
                        ).toLocaleDateString(
                            "hu-HU"
                        )
                    }
                </p>

                <p>
                    <strong>
                        Fizetési mód:
                    </strong>{" "}
```

```
        {
            fizetes.fizetesmod
        }
    </p>

    {
        fizetes.megjegyzes &&
        (
            <p>
                <strong>
                    Megjegyzés:
                </strong>{" "}
                {
                    fizetes.megjegyzes
                }
            </p>
        )
    }

    <p>

        <strong>
            Státusz:
        </strong>

        <span
            className={
                fizetes.statusz === "Teljesítve"
                ? "active-status"
                : fizetes.statusz === "Függőben"
                ? "pending-status"
                : "expired-status"
            }
        >
            {" "}
            {
                fizetes.statusz
            }
        </span>

    </p>

    <div className="buttons">

        <button
            type="button"
            className="edit-btn"
            onClick={() => {
```

```

        setSzerkesztettId(
            fizetes.id
        );

        setUjFizetes({
            szerzodesid:
                fizetes.szerzodesid,
            osszeg:
                fizetes.osszeg,
            fizetesdatum:
                fizetes.fizetesdatum?.split("T")[0] ||
                fizetes.fizetesdatum,
            statusz:
                fizetes.statusz,
            fizetesmod:
                fizetes.fizetesmod,
            megjegyzes:
                fizetes.megjegyzes || ""
        });

        window.scrollTo({
            top: 0,
            behavior: "smooth"
        });

    }}
>
    Módosítás
</button>

<button
    type="button"
    className="delete-btn"
    onClick={() =>
        torles(
            fizetes.id
        )
    }
>
    Törlés
</button>

</div>

</div>

)

```

```
        )}

    </div>

</div>

);

}

export default Fizetesek;
```

2. fizeteseklistazasa.js

Megvalósítottam a fizetések listázását a backend API végpont használatával. A lekérdezett adatokat React állapotkezelés segítségével tároltam és jelenítettem meg. A fizetéseket kártyás elrendezésben jelenítettem meg a könnyebb áttekinthetőség érdekében. Minden kártyán feltüntettem a bérlő nevét, az összeget, a dátumot és a fizetés módját. A listázás során figyeltem arra, hogy az adatok naprakészen jelenjenek meg minden művelet után.

```
const express = require("express");
const router = express.Router();

const pool = require("../db");

router.get("/", async (req, res) => {

    let conn;

    try {

        conn = await pool.getConnection();

        const fizetesek = await conn.query(`
            SELECT
                fizetesek.id,
                fizetesek.szerzodesid,
                fizetesek.osszeg,
                fizetesek.fizetesdatum,
                fizetesek.statusz,
                fizetesek.fizetesmod,
                fizetesek.megjegyzes,

                berlok.id AS berloid,
                berlok.nev AS berlonev,
                berlok.email AS berloemail,
                berlok.telefonszam,
```

```
        szerzodesek.ingatlanid,  
        szerzodesek.kezdetdatum,  
        szerzodesek.vegdatum,  
        szerzodesek.havidij,  
        szerzodesek.statusz AS szerzodesstatusz  
  
    FROM fizetesek  
  
    INNER JOIN szerzodesek  
        ON szerzodesek.id = fizetesek.szerzodesid  
  
    INNER JOIN berlok  
        ON berlok.id = szerzodesek.berloid  
  
    ORDER BY fizetesek.fizetesdatum DESC  
`);  
  
res.status(200).json(fizetesek);  
  
} catch (err) {  
  
    console.error("Fizetések lekérdezési hiba:", err);  
  
    res.status(500).json({  
        siker: false,  
        uzenet: "Hiba történt a fizetések lekérdezése során!"  
    });  
  
} finally {  
  
    if (conn) {  
        conn.release();  
    }  
  
}  
  
});  
  
module.exports = router;
```

3. fizeteselektréhozasa.js

Kialakítottam egy adatbeviteli űrlapot új fizetések rögzítéséhez. Az űrlapon megadható a szerződés, az összeg, a fizetés dátuma, a státusz és a fizetési mód. Az adatok elküldése POST kérés segítségével történik a backend felé. A sikeres mentés után visszajelzést jelenítettem meg a felhasználó számára.

```
const express = require("express");
```



```
const router = express.Router();

const pool = require("../db");

router.post("/", async (req, res) => {

  const {
    szerzodesid,
    osszeg,
    fizetesdatum,
    statusz,
    fizetesmod,
    megjegyzes
  } = req.body;

  let conn;

  try {

    conn = await pool.getConnection();

    await conn.query(
      `INSERT INTO fizetesek
      (szerzodesid, osszeg, fizetesdatum, statusz, fizetesmod, megjegyzes)
      VALUES (?, ?, ?, ?, ?, ?)`,
      [
        szerzodesid,
        osszeg,
        fizetesdatum,
        statusz,
        fizetesmod,
        megjegyzes
      ]
    );

    res.json({
      siker: true,
      uzenet: "Fizetes sikeresen rogzitve!"
    });

  } catch (err) {

    res.status(500).json(err);

  } finally {

    if(conn) conn.release();

  }

});
```

```
    }  
  
  });  
  
module.exports = router;
```

4. fizetesekmodositasa.js

Lehetővé tettem a korábban rögzített fizetések módosítását. A módosítás gombra kattintva a kiválasztott rekord adatait visszatöltöttem az űrlap mezőibe. A felhasználó ezután szerkesztheti az adatokat, majd elmentheti a változtatásokat. A módosított adatokat PUT kérés segítségével továbbítottam a backendnek. A mentés után a lista automatikusan frissül, így az új adatok azonnal láthatóvá válnak.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.put("/:id", async (req, res) => {  
  
  const {  
    osszeg,  
    fizetesdatum,  
    statusz,  
    fizetesmod,  
    megjegyzes  
  } = req.body;  
  
  let conn;  
  
  try {  
  
    conn = await pool.getConnection();  
  
    await conn.query(  
      `UPDATE fizetesek  
      SET  
      osszeg = ?,  
      fizetesdatum = ?,  
      statusz = ?,  
      fizetesmod = ?,  
      megjegyzes = ?  
      WHERE id = ?`,  
      [  
        osszeg,  
        fizetesdatum,  
        statusz,
```

```
        fizetesmod,  
        megjegyzes,  
        req.params.id  
    ]  
    );  
  
    res.json({  
        siker: true,  
        uzenet: "Fizetés módosítva!"  
    });  
  
    } catch (err) {  
  
        res.status(500).json(err);  
  
    } finally {  
  
        if(conn) conn.release();  
  
    }  
});  
  
module.exports = router;
```

5. fizetesektorlese.js

Megvalósítottam a fizetések törlésének funkcióját is. A törlés előtt megerősítő ablakot jelenítettem meg, hogy elkerüljem a véletlen adatvesztést. A rekord eltávolítása DELETE kérés segítségével történik. A backend sikeres válasza után értesítést jelenítettem meg a felhasználónak. A törlés befejezése után újra lekérdeztem az adatokat, hogy a lista naprakész maradjon.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.delete("/:id", async (req, res) => {  
  
    let conn;  
  
    try {  
  
        conn = await pool.getConnection();  
  
        await conn.query(  
            "DELETE FROM fizetesek WHERE id = ?",  

```

```
        [req.params.id]
    );

    res.json({
        siker: true,
        uzenet: "Fizetés törölve!"
    });

} catch (err) {

    res.status(500).json(err);

} finally {

    if(conn) conn.release();

}

});

module.exports = router;
```

fizetesek.css

Elkészítettem a fizetések oldalhoz tartozó CSS stíluslapot, amely biztosítja a felület egységes és modern megjelenését. A fő konténert úgy alakítottam ki, hogy a tartalom középre igazítva, megfelelő margókkal és térközökkel jelenjen meg. Az adatbeviteli űrlapot rácsszerkezet segítségével készítettem el, így különböző képernyőméreteken is jól használható marad. A gombokhoz és mezőkhöz lekerekített sarkokat, valamint lebegéskor megjelenő színváltást alkalmaztam a jobb felhasználói élmény érdekében.

```
.tenant-container {
    padding: 30px;
    max-width: 1400px;
    margin: auto;
}

.tenant-container h1 {
    text-align: center;
    margin-bottom: 25px;
}

.tenant-form {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 15px;
    margin-bottom: 30px;
}

.tenant-form input,
```

```
.tenant-form select {
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 8px;
  font-size: 14px;
}

.tenant-form button {
  background: #2563eb;
  color: white;
  border: none;
  border-radius: 8px;
  padding: 12px;
  cursor: pointer;
  transition: 0.3s;
}

.tenant-form button:hover {
  background: #1d4ed8;
}

.filter-container {
  margin-bottom: 25px;
}

.filter-container select {
  padding: 10px;
  border-radius: 8px;
  border: 1px solid #ccc;
}

.tenant-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(320px, 1fr));
  gap: 20px;
}

.tenant-card {
  background: white;
  border-radius: 12px;
  padding: 20px;
  box-shadow: 0px 2px 10px rgba(0,0,0,0.1);
}

.tenant-card h3 {
  margin-bottom: 15px;
  color: #2563eb;
}
```

```
.tenant-card p {
  margin: 8px 0;
}

.buttons {
  display: flex;
  gap: 10px;
  margin-top: 15px;
}

.edit-btn {
  flex: 1;
  background: #f59e0b;
  color: white;
  border: none;
  padding: 10px;
  border-radius: 8px;
  cursor: pointer;
}

.edit-btn:hover {
  background: #d97706;
}

.delete-btn {
  flex: 1;
  background: #dc2626;
  color: white;
  border: none;
  padding: 10px;
  border-radius: 8px;
  cursor: pointer;
}

.delete-btn:hover {
  background: #b91c1c;
}

.active-status {
  color: green;
  font-weight: bold;
}

.expired-status {
  color: red;
  font-weight: bold;
}
```

```
@media (max-width: 768px) {  
  
  .tenant-form {  
    grid-template-columns: 1fr;  
  }  
  
  .tenant-grid {  
    grid-template-columns: 1fr;  
  }  
  
}
```

The screenshot shows a web application interface for managing payments. The main heading is 'Fizetések'. Below it is a form with five input fields: a dropdown for 'Válassz szerződést', a text field for 'Összeg', a date field for 'éééé. hh. nn.', a dropdown for 'Teljesítve', and a text field for 'Fizetési mód'. There are two buttons: 'Megjegyzés' and 'Fizetés rögzítése'. Below the form is a section for 'Összes fizetés' with a dropdown. A card displays payment details for 'Biró Zoltán': 'Összeg: 14500.00 Ft', 'Dátum: 2026. 06. 19.', 'Fizetési mód: Bankkártya', 'Megjegyzés: Új fizetési kötelezettség!', and 'Státusz: Független'. At the bottom of the card are buttons for 'Módosítás' and 'Törölés'.

1.Kép: Fizetések HTML oldala a funkciókkal ellátva.

Üzenetek funkciók létrehozása és beállítása

Üzenetek.jsx

Az Uzenetek.jsx komponensben kialakítottam az üzenetkezelő felhasználói felületet. Betöltöm az adatbázisban tárolt üzeneteket, majd azokat kártyás elrendezésben jelenítem meg a felhasználók számára. Lehetővé teszem új üzenetek küldését egy űrlapon keresztül, ahol megadható az üzenet tárgya és tartalma. A kitöltött adatokat a backend API segítségével továbbítom az adatbázis felé. Sikeres küldés után automatikusan frissítem az üzenetek listáját.

```
import { useEffect, useState } from "react";  
import "../styles/uzenetek.css";  
  
function Uzenetek() {  
  
  const user =  
    JSON.parse(localStorage.getItem("user"));
```

```
const [uzenetek, setUzenetek] = useState([]);

const [ujUzenet, setUjUzenet] = useState({
  targy: "",
  uzenet: ""
});

const lekeres = async () => {

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/uzenetlistazas"
    );

    const adatok = await valasz.json();

    setUzenetek(adatok);

  } catch (err) {

    console.error(err);

  }

};

useEffect(() => {

  lekeres();

}, []);

const kuldes = async (e) => {

  e.preventDefault();

  try {

    const valasz = await fetch(
      "http://localhost:3001/api/uzenetkuldes",
      {
        method: "POST",
        headers: {
          "Content-Type":
            "application/json"
        },
      },
    );

  }

};
```



```
        body: JSON.stringify({
            kuldoid: user.id,
            targy: ujUzenet.targy,
            uzenet: ujUzenet.uzenet
        })
    }
});

const adat = await valasz.json();

console.log(adat);

alert(
    adat.uzenet ||
    "Üzenet sikeresen elküldve!"
);

setUjUzenet({
    targy: "",
    uzenet: ""
});

lekeres();

} catch (err) {

    console.error(err);

    alert("Hiba történt!");

}

};

return (

    <div className="messages-container">

        <h1>Üzenetek</h1>

        <form
            className="message-form"
            onSubmit={kuldes}
        >

            <input
                type="text"
                placeholder="Tárgy"
```

```
        value={ujUzenet.targy}
        onChange={(e) =>
          setUjUzenet({
            ...ujUzenet,
            targy: e.target.value
          })
        }
        required
      />

      <textarea
        placeholder="Üzenet"
        value={ujUzenet.uzenet}
        onChange={(e) =>
          setUjUzenet({
            ...ujUzenet,
            uzenet: e.target.value
          })
        }
        required
      />

      <button type="submit">
        Üzenet küldése
      </button>

    </form>

    <div className="messages-grid">

      {uzenetek.map((uzenet) => (

        <div
          className="message-card"
          key={uzenet.id}
        >

          <h3>
            {uzenet.targy}
          </h3>

          <p>
            <strong>
              Feladó:
            </strong>{" "}
            {uzenet.nev || "Ismeretlen"}
          </p>

        )
      )
    }

  )
}
```

```
        <p>
            <strong>
                E-mail:
            </strong>{" "}
            {uzenet.email || "-"}
        </p>

        <p>
            <strong>
                Üzenet:
            </strong>
        </p>

        <div className="message-text">
            {uzenet.uzenet}
        </div>

        <p className="message-date">

            {
                uzenet.kuldesdatum
                ? new Date(
                    uzenet.kuldesdatum
                ).toLocaleString(
                    "hu-HU"
                )
                : "-"
            }

        </p>

    </div>

    )})

</div>

</div>

);

}
```

```
export default Uzenetek;
```

uzenetek.css

Az uzenetek.css fájlban kialakítottam az üzenetkezelő oldal teljes megjelenését. A stíluslap segítségével középre igazítottam a tartalmat, valamint modern és áttekinthető elrendezést hoztam létre az üzenetküldő űrlap és az

üzenetkártyák számára. Az űrlapelemekhez és gombokhoz egységes megjelenést, lekerekített sarkokat és árnyékokat alkalmaztam a felhasználói élmény javítása érdekében. Az üzenetek megjelenítésére rugalmas rácsszerkezetet használtam, amely automatikusan alkalmazkodik a rendelkezésre álló képernyőmérethez.

```
.messages-container {  
  
    max-width: 1200px;  
    margin: auto;  
    padding: 20px;  
  
}  
  
.messages-container h1 {  
  
    text-align: center;  
    margin-bottom: 25px;  
  
}  
  
.message-form {  
  
    display: flex;  
    flex-direction: column;  
    gap: 15px;  
  
    background: white;  
    padding: 20px;  
    border-radius: 12px;  
  
    box-shadow:  
        0 2px 10px rgba(0,0,0,0.1);  
  
    margin-bottom: 30px;  
  
}  
  
.message-form input,  
.message-form textarea {  
  
    padding: 12px;  
    border: 1px solid #ccc;  
    border-radius: 8px;  
  
}  
  
.message-form textarea {  
  
    min-height: 120px;
```

```
        resize: vertical;
    }

.message-form button {

    background: #2563eb;
    color: white;

    border: none;
    padding: 12px;

    border-radius: 8px;
    cursor: pointer;
}

.message-form button:hover {

    background: #1d4ed8;
}

.messages-grid {

    display: grid;

    grid-template-columns:
        repeat(auto-fill,
            minmax(320px, 1fr));

    gap: 20px;
}

.message-card {

    background: white;

    padding: 20px;

    border-radius: 12px;

    box-shadow:
        0 2px 10px rgba(0,0,0,0.1);
}
```

```
.message-card h3 {  
  
    margin-bottom: 15px;  
    color: #2563eb;  
  
}  
  
.message-text {  
  
    margin-top: 10px;  
    margin-bottom: 15px;  
  
    padding: 10px;  
  
    background: #f5f5f5;  
  
    border-radius: 8px;  
  
}  
  
.message-date {  
  
    color: gray;  
    font-size: 14px;  
    text-align: right;  
  
}
```

uzenetkuldes.js

Az `uzenetkuldes.js` fájlban megvalósítottam az új üzenetek rögzítését. A POST végpont fogadja a felhasználó által elküldött adatokat, például a küldő azonosítóját, az üzenet tárgyát és szövegét. Ellenőrzöm, hogy a szükséges adatok rendelkezésre állnak-e, majd INSERT lekérdezéssel eltárolom azokat az adatbázisban. Sikeres mentés esetén visszajelzést küldök a kliens számára az üzenet elküldéséről. Hiba esetén részletes hibaüzenetet és megfelelő HTTP választ adok vissza.

```
const express = require("express");  
const router = express.Router();  
  
const pool = require("../db");  
  
router.post("/", async (req, res) => {  
  
    const {  
        kuldoid,  
        targy,  
        uzenet  
    } = req.body;
```

```
let conn;

try {

  console.log("Beérkezett adat:", req.body);

  if (!kuldoid) {

    return res.status(400).json({
      siker: false,
      uzenet: "Hiányzik a kuldoid!"
    });

  }

  conn = await pool.getConnection();

  const eredmeny = await conn.query(
    `INSERT INTO uzenetek
    (
      kuldoid,
      targy,
      uzenet
    )
    VALUES (?, ?, ?)` ,
    [
      kuldoid,
      targy,
      uzenet
    ]
  );

  console.log("INSERT eredmény:", eredmeny);

  res.json({
    siker: true,
    uzenet: "Üzenet elküldve!"
  });

} catch (err) {

  console.error("Hiba:", err);

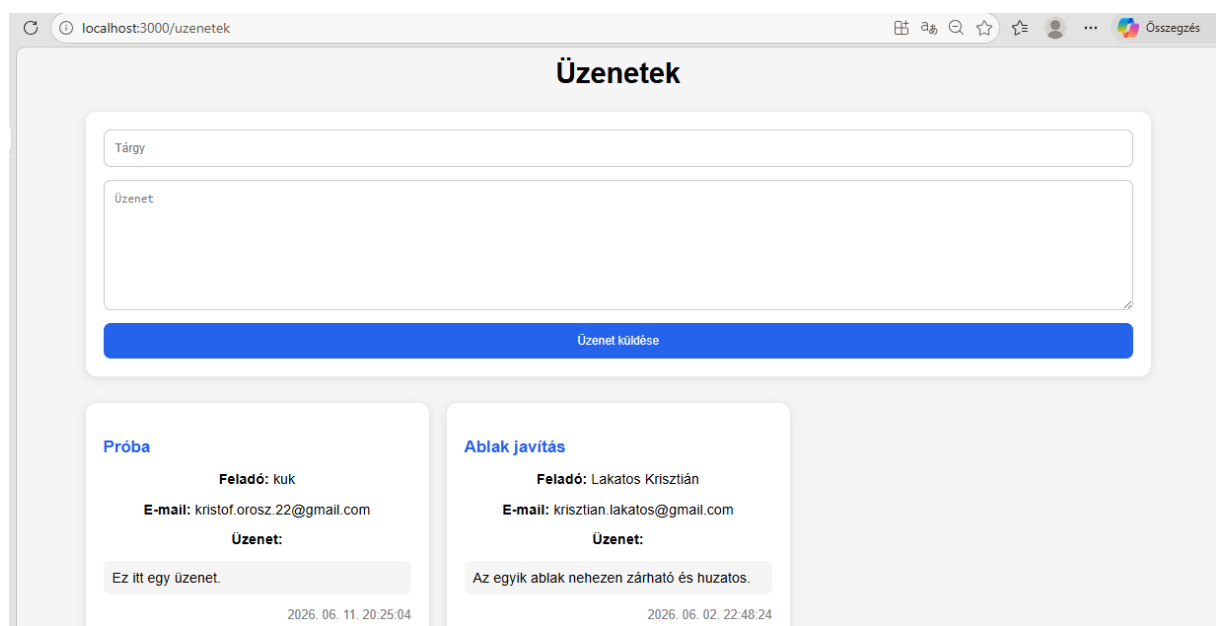
  res.status(500).json({
    siker: false,
    uzenet: "Szerverhiba történt!",
    hiba: err.message
  });

}
```

```
    } finally {  
  
        if (conn) {  
            conn.release();  
        }  
  
    }  
  
});  
  
module.exports = router;
```

uzenetlistazas.js

Az uzenetlistazas.js fájlban létrehoztam egy Express végpontot az üzenetek lekérdezésére. Kapcsolatot létesíték az adatbázissal, majd SQL lekérdezéssel összegyűjtöm az üzeneteket és a hozzájuk tartozó felhasználói adatokat. Az adatok összekapcsolására INNER JOIN műveletet alkalmazok a users és az uzenetek táblák között. A lekérdezés eredményét JSON formátumban küldöm vissza a frontend számára. Hiba esetén naplózom a problémát és megfelelő hibakódot küldök vissza.



1.Kép: Üzenetek HTML oldal megjelenítése és űrlap megjelenítése.